A WALKTHROUGH · 28 MAY 2026

See what an attacker would do next.

K8s Sentinel scans your Kubernetes cluster the way an attacker would think about it — not as a flat list of CVEs, but as a graph of paths. It tells you which single fix breaks the most ways in. It never touches your cluster.

Version	Status	Dashboard	Source
0.2.0 · ARGUS v3	Live in production	control-plane-azure.vercel.app	github.com/evanapple83-png/k8s-sentinel

01 What it does

Three things, in plain English.

Maps the attack surface

Every workload, service account and exposure path in your cluster gets drawn into a single graph: who can reach what, and how.

Ranks what to fix first

A finding only matters if an attacker can *reach* it. We score by reachability — not raw CVSS — and surface the one or two fixes that collapse the most paths.

Proposes, never applies

Fixes ship as reviewable pull-request bundles for your repo. Sentinel has read-only credentials, by construction.

Speaks plain language

Ask *"show everything internet-exposed running as root"* in the dashboard and get the answer in one sentence and a link.

How it's different

Most scanners give you a list of CVEs sorted by severity. That list lies. A "critical" vulnerability on a workload no one can reach is noise; a "medium" on the internet-exposed payment API with a service account that can read every Secret in the cluster is a fire. Sentinel knows the difference because it builds the graph first and scores second.

> The headline isn't *"you have 412 vulnerabilities."*
> It's *"apply this one patch and 4 of 4 active attack paths collapse."*

02 The dashboard

Six screens. Each one answers a question you already had.

SCREEN	THE QUESTION IT ANSWERS
Overview	How bad is it today, and what should I do first?
Findings	Which vulnerabilities actually matter — and why?
Attack Paths	If an attacker got in here, where would they end up?
Ask	Anything else. Type a question, get a sentence.
Report	Give me the audit-ready PDF / MD / JSON.
Fixes	What can I approve right now? (One click → PR bundle.)

On the Overview, four numbers

Critical · KEV · SSVC Act · Reachable. The first one is the count of "high severity" issues — useful, but it's just the loudest number. The other three are what you actually act on.

KEV

Listed in CISA's Known-Exploited Vulnerabilities catalogue. Someone, somewhere, is using this in the wild today. Patch on sight.

SSVC Act

The vulnerability is being exploited *and* reaches a crown jewel in your cluster (a Secret, cluster-admin, a cloud identity). The smallest, scariest, most useful pile.

Reachable

At least one path exists from the public internet — or from a less- trusted namespace — to this finding. If it's not reachable, it's noise.

"Apply first" panel

A short, ranked list of single controls that break the most paths. Usually it's two or three items, and usually number one is a patch.

03 Connect a cluster

Two methods. Both end at the same dashboard. Pick whichever fits your week, not your beliefs.

Helm install

Best for clusters you'll keep scanning. The agent runs *inside* your cluster and dials out to our relay — no inbound port, no firewall hole, works behind air-gap.

1. **Mint a token** in the dashboard's Connect screen.
2. **Run one Helm command** on a host with cluster-admin.
3. **Wait fifteen seconds.** The page flips to *connected*.

Public key

Best for a quick first look, or admins who don't want to deploy an agent. Your cluster signs a short-lived, read-only certificate it explicitly approves. The private key never leaves your machine.

1. **Mint a token**, same as Helm.
2. **Run one CLI command**: keypair + CSR happen locally.
3. **Approve the certificate** with the exact `kubectl` line we print for you. The page flips to *connected*.

The Helm one-liner

```
helm install sentinel deploy/helm \
  --namespace sentinel --create-namespace \
  --set mode=hybrid \
  --set relay.url=wss://k8s-sentinel-relay.fly.dev \
  --set relay.installTokenSecret=sentinel-install
```

The Public-key one-liner

```
python3 -m argus.cli bootstrap csr \
  --enroll ent_<token-from-UI> \
  --control-plane https://control-plane-azure.vercel.app \
  --auto-approve --cleanup    # auto-approve only on kind/dev
```

> Neither method ever sees your `kubeconfig`. Helm uses an in-cluster service-account token. Public-key uses a fresh certificate that expires in one hour by default. Your real admin credentials stay where they are.

---

04 What scans your cluster

The agent runs four open-source scanners in parallel, then feeds them into one engine that thinks about *reachability*.

T

Trivy

Looks inside every container image for known CVEs. We pin `0.70.0` — bumps are deliberate supply-chain decisions, not accidents.

K

Kubescape

Checks your manifests against NSA-CISA and MITRE frameworks. Catches the configuration mistakes that turn a CVE into a breach.

B

kube-bench

Runs the CIS Kubernetes Benchmark against your control plane and nodes. Boring, essential, exactly what your auditor will ask about.

F

Falco

Watches runtime behaviour — a shell opening in production, a process reading `/etc/shadow`. Optional and additive.

And one engine on top: ARGUS v3

The scanners are commodity. The interesting bit is what we do with their output. ARGUS pulls down the live **CISA Known-Exploited Vulnerabilities** catalogue at scan time, fuses every finding with your live RBAC, exposed services, network policies and pod volume mounts, and produces a single attack graph.

Each path in that graph is then scored with the **SSVC** decision model — the same one CISA's own coordinators use:

SSVC TIER	PLAIN ENGLISH
Act	Exploited *and* reaches a crown jewel today. Drop everything.

SSVC TIER	PLAIN ENGLISH
Attend	Exploited, but the path is interrupted by an existing control. Plan a sprint.
Track	Watched but quiet. Stay informed; no rush.
Track*	No reachable impact. Inventory only.

The dashboard's filters mirror this — *SSVC Act* is one click, and that's where you start.

05 Apply this first

The single best feature: telling you which one fix breaks the most ways in.

Every attack path the engine finds is a chain — usually four or five steps long. Most steps have a control that, if applied, breaks the chain. ARGUS counts: *"if you applied this one control, how many active paths would collapse?"* The "Apply first" card on the Overview is just that list, sorted descending.

A real example, redacted

> **Patch CVE-2026-31337 on prod/invoice-api** · breaks 4 of 4 active paths.
>
> Eliminates → secret:payments/db-credentials · secret:ci/registry-token · CLUSTER-ADMIN · CLOUD-ADMIN

One patch on one workload, and four crown-jewel access paths close. You don't get to that conclusion from a CVE list. You get to it from a graph plus a counter.

The second pattern: RBAC tightening

After patches, the most common choke-point we surface is removing an over-broad *service account → Secrets* binding. A workload that doesn't need to read Secrets shouldn't be able to. ARGUS flags every binding where the privilege exceeds the workload's actual reach.

What ships in the Fixes screen

1. A **ranked list** of remediations — patches, RBAC tightening, network policies, security-context hardening — each with a one-line rationale.
2. A **full unified diff** against your manifests where applicable.
3. A **ready-to-open PR body** with the engine's reasoning, the controls it references, and the findings it closes.
4. An **Approve** button that writes the PR bundle to disk for review. *Nothing is ever applied to your cluster.*

06 Reports & conversation

The artefact your auditor wants. The answer your CTO wants. The explanation your on-call wants.

Reports — four formats, one click

PDF

Apple-styled, paginated, dependency-free generator. The format your auditor will accept without follow-up questions.

Markdown

Drop it in your platform repo, in a Confluence page, in a Slack thread. Renders everywhere.

JSON

The full normalized posture for downstream tools — SIEMs, ticketing, your own dashboards.

HTML

The same dashboard view, packaged as a single file. Forward it; it stands alone.

Ask — the Spotlight bar for security

The *Ask* screen is one input box. Type a plain-English question; the analyst answers in one sentence, with the matching resources linked. No fluff, no opinion drift — it operates only on the data the engine already collected, never reaches outside.

```
"Show everything internet-exposed running as root."
"Which service accounts can read Secrets cluster-wide?"
"What changed since the last scan?"
"Why is finding trivy-001 ranked above kubescape-005?"
```

Behind the scenes, queries are sanitized at the trust boundary before any LLM sees them. Scan input is hardened the same way — there is no path by which an attacker who controls a container image annotation can make the assistant generate exploit code. Defensive by construction.

07 Six promises

The things we will not do, in any mode, ever.

1

Read-only by default.

The agent's Kubernetes role grants only `get`, `list`, `watch`. There is no way through this product to widen it.

2

Zero *secrets* verbs.

We don't list Secret names. We don't read Secret data. Ever. Secret *reachability* is inferred from RBAC + volume mounts.

3

Propose, don't apply.

Every fix is a pull-request bundle for a human to approve. The agent cannot, by construction, change your cluster or your repo.

4

Untrusted input is hardened.

CVE descriptions, image tags, scanner output — all treated as hostile before they reach an LLM. One sanitisation chokepoint, audited, tested.

5

Everything is logged.

Hash-chained immutable audit. Every decision, every tool call, every command leaves a verifiable trail.

6

Air-gap is one flag.

Switch the LLM engine to the local *Hermes* adapter and the chart drops all public egress. Same product, zero external calls.

08 Get started

Sixty seconds to a live scan against a real cluster.

1. **Open the dashboard.**
   `https://control-plane-azure.vercel.app`
2. **Sign in** (GitHub OAuth) and open *Connect a cluster.*
3. **Pick a method.** Helm for the long term; Public-key for the next five minutes.
4. **Run the one command** we print for you, on any machine with cluster-admin access.
5. **Click *Run scan*.** First scan finishes in 30 – 90 seconds.
6. **Read the Overview.** Apply the top "Apply first" item this week.

Going to production

For ongoing use, install the Helm chart in `hybrid` mode (agent in your cluster, dashboard hosted by us) or `cluster-local` mode for air-gap. Add the accepted-risk waiver ConfigMap if you have findings you've explicitly chosen to live with — the engine will keep those out of your action list, but auto-reopen if the underlying control weakens.

Where the docs live

DOC	FOR
README.md	Overview + quick links
docs/GO_LIVE.md	Image build → Supabase migration → cluster onboard
docs/BUILD.md	The original product spec (architecture, phases, security model)
docs/DATA-BOUNDARY.md	Exactly which bytes ever leave a cluster
docs/PUBKEY_CONNECT_SPEC.md	Public-key method, design notes
deploy/helm/README.md	Every chart value, both modes

> The goal is uncomplicated: tell you what an attacker would do today, what to fix this week, and never touch your cluster doing it.
